

Fashion Garment Classification & Inspiration Web App

Build a lightweight AI-powered web app that helps fashion designers organize, search, and reuse inspiration imagery captured in the field.

Overview: Design teams often collect thousands of inspiration photos from stores, markets, and streetwear across geographies. When those images remain scattered across phones and shared folders, they become difficult to organize, search, compare, and build on. This exercise explores how AI can turn that image library into a usable, searchable source of inspiration.

The Challenge

Create a simple web app where designers can upload photos, automatically classify garments using AI, search and filter across fashion and contextual metadata, and add their own annotations over time.

Users

Fashion designers who regularly capture inspiration images in the field and need a fast, intuitive way to organize and revisit them.

What to Build

1. Image Upload + AI Classification

- Users can upload garment photos through the web app.
- Each image should be processed by a multimodal AI model that returns both a rich natural-language description and structured attributes such as garment type, style, material, color palette, pattern, season, occasion, consumer profile, trend notes, and location context.
- Store both the descriptive output and the structured metadata.

2. Search + Filtering

- Display the image library in a visual grid.
- Support filtering across garment attributes such as garment type, style, material, color palette, pattern, occasion, consumer profile, and trend notes.
- Support contextual filters including location (continent, country, city), time (year, month, season), and designer.
- Filters should be dynamically generated from the data rather than hardcoded.
- Support full-text search across image descriptions using natural queries such as "embroidered neckline" or "artisan market."

3. Designer Annotations

- Allow users to add their own tags, notes, and observations in addition to AI-generated metadata.
- Annotations should be searchable and clearly distinguished from AI-generated output.

Deliverables

Area	Expectation
Implementation	Build the web app using any stack you prefer. A lightweight full-stack implementation is completely acceptable. The submission should run locally with minimal setup and include a clear README with setup instructions and key architectural choices.
Model Evaluation	Use a test set of at least 50-100 garment images or street fashion images selected from any open-source fashion image dataset of your choice. For example : https://www.pexels.com/search/fashion/ Define expected attributes manually, run your classifier against the test set, report per-attribute accuracy for fields such as garment type, style, material, occasion, and location context, and briefly explain where the model performs well, where it struggles, and how you would improve it with more time.
Testing	Include a unit test for parsing model output into structured attributes, an integration test for filter behavior (especially location and time filters), and at least one end-to-end test covering upload, classify, and
Repository Structure	/app - application source /eval - evaluation script and labeled test set /tests - unit, integration, and end-to-end tests README.md - setup, architecture notes, and evaluation summary Please commit in logical increments. We care about how you structure and communicate your work.

What We'll Evaluate

- Functionality - Does the core workflow work end to end?
- Model quality - How well does the classifier perform, and how did you evaluate it?
- Code quality - Is the code well-structured, readable, and appropriately tested?
- Product thinking - Did you make sensible trade-offs for a proof of concept?
- Communication - Is the README clear, thoughtful, and honest about limitations?

Guidance

- Please timebox this to roughly within a day.
- Simplifying assumptions are completely fine; document them clearly. You may use AI tools as part of your workflow.
- If something is incomplete, call it out and describe what you would do next.